String Matching Project Report

Abstract

This document presents the research, implementation, and analysis of four fundamental string-matching algorithms: the Naive (Brute-Force) approach, Rabin-Karp, Knuth-Morris-Pratt (KMP), Boyer-Moore and Aho-Corasick. A command-line tool was developed to search for keywords within a 2D character grid in both horizontal and vertical directions. The report evaluates the theoretical complexity and practical performance of these algorithms.

Index Terms

string matching, brute-force, rabin-karp, kmp, boyer-moore, aho-corasick, algorithm analysis

I. Group Information

TABLE I
GROUP INFORMATION AND ASSIGNED TASKS

Student Name	Student ID	Assigned Tasks	% Completed
Dao Hong Dang	25127026	*Brute-Force Research, Algorithm Implementation, Source Code Organization*	100%
Nguyen Quoc Dat	25127033	*Rabin-Karp Research, Algorithm Implementation, Demo Video Recording*	100%
Tran Duc Hien	25127048	*Aho-Corasick Research, Testing and Step-by-step Trace, Documentation (Readme)*	100%
Pham Bach Dang Khoa	25127202	*Boyer-Moore Research, Experiments and Metric, Chart Generation*	100%
Tran Gia Bao	25127283	*KMP Research, Report Drafting, IEEE Formatting*	100%
		Total Group Completion	**100%**

II. Problem Statement

The string-matching problem is one of the most fundamental and important issues in computer science. Although this operation seems as simple and familiar as the ”Find” function in text editors, it holds immense practical significance, being widely applied in areas ranging from DNA sequence analysis in bioinformatics to plagiarism detection in academic documents.

Within the scope of this project, the problem requires designing and building a command-line tool to automate string matching and keyword searching. Specifically, the problem is defined by the following elements:

- **Input:** The program will read a text-based grid of characters and a list of keywords provided by the user.
- **Output:** Find and output the exact coordinates (start and end positions) of those keywords within the grid if they exist.
- **Search Space:** The program must support searching in two spatial directions: horizontal (scanning from Left-to-Right) and vertical (scanning from Top-to-Bottom).

To solve this problem, our group researches, implements, and compares the performance of four core string-matching algorithms: the Naive (Brute-Force) approach, Rabin-Karp, Knuth-Morris-Pratt (KMP), and Boyer-Moore. By measuring execution times, we aim to clarify the advantages and disadvantages of each method when handling various grid sizes and keyword set dimensions.

III. Algorithm Analysis

1) Brute-Force (Naïve) Algorithm:

a) Introduction & Application: In this algorithm, we perform the search exactly as the basic definition implies. To find a pattern P in a text T, we sequentially iterate through the text. For each starting position s, we check if all characters of P perfectly match the subsequent characters in T. If a full match is found, we record the result starting at s. If a mismatch occurs at any point, we immediately stop checking the current window and move to the next starting position $s + 1$. Because of its simplicity, this approach requires no preprocessing and is often used as a baseline or in standard library functions for extremely short strings.

b) Pseudocode:

```
n = T.length
m = P.length
for s = 0 TO n − m do
  match = true
  for i = 0 TO m − 1 do
    if P[i] ≠ T[s + i] then
      match = false
      break
    end if
  end for
  if match == true then
    print "Pattern occurs with shift " s
  end if
end for
```

c) Step-by-step trace: To graphically demonstrate the exhaustive nature of the Naïve algorithm, we trace its execution on the running example with Text $T = \texttt{ababc}$ and Pattern $P = \texttt{abc}$ (where $m = 3$). Figure 1 illustrates how the pattern is shifted by exactly one position after every mismatch.

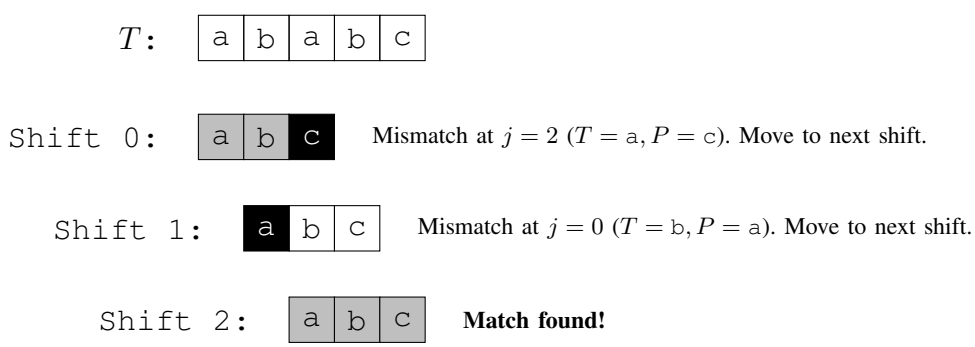

Fig. 1. Visual trace of the Naïve (Brute-Force) algorithm. At each shift, the pattern is compared character by character from left to right until a mismatch occurs. Regardless of how many characters matched, the window strictly shifts by 1 position.

d) Complexity:

- **Time Complexity:** $O((n - m + 1)m)$ in the worst case, where n is the length of the text and m is the length of the pattern.
- **Space Complexity:** $O(1)$ because no extra arrays or preprocessing are required.

e) Discussion: This algorithm is **good** (best-case scenario) when the text or pattern is very short, or when a mismatch occurs immediately at the first character for almost every shift. For example, if $T = \texttt{bbbbbbb}$ and $P = \texttt{a}$, the algorithm detects a mismatch at P instantly and shifts, yielding a fast $O(n)$ time.

Conversely, it is **bad** (worst-case scenario) when the text and pattern contain highly repetitive characters. For instance, if $T = \texttt{aaaa...aaa}$ and $P = \texttt{aaaaa}$, the algorithm successfully matches the first 4

characters but either matches or fails at the very last character of P. Because it must naively slide the window by just 1 position and re-evaluate everything from scratch, the number of redundant character comparisons will be astronomically high, degrading the performance to the worst-case $O((n-m+1)m)$ time.

f) Conclusion: This algorithm is **good** when the text or pattern is very short, or when the alphabet size is large resulting in mismatches happening mostly at the first character (bringing the average time to nearly $O(n)$). It is **bad** (worst-case scenario) when the text has many repetitive elements and the mismatch always occurs at the very last character of the pattern, forcing the algorithm to do redundant comparisons.

g) Implementation Reference & Optimization: The source code for our Brute-Force implementation is directly referenced and adapted from the pseudocode provided in the textbook *Introduction to Algorithms* by Cormen et al. [1]. Because this algorithm inherently relies on exhaustive character-by-character comparison, the implementation strictly adheres to the standard theoretical approach. No further architectural optimizations were applied, as the baseline code is already optimally straightforward for what the algorithm requires.

2) Rabin-Karp Algorithm:

a) Introduction & Application: The Rabin-Karp algorithm improves upon the Naïve approach by utilizing a rolling hash function. Instead of checking every character at every shift, it computes a hash value for the pattern and for each window of the text. A full character-by-character comparison is only performed if the hash values match. This algorithm is highly effective for multiple pattern matching and is widely used in real-world applications such as plagiarism detection systems.

Real-world Application: The Rabin-Karp algorithm is famously known for its core role in **plagiarism detection systems** (such as MOSS). Because its rolling hash mechanism can easily be extended using a Hash Set, it is exceptionally efficient at multiple-pattern matching—allowing it to simultaneously search for many different suspicious phrases within a massive academic document.

b) Pseudocode:

```
n = T.length, m = P.length
h = d^(m-1) (mod q)
p = 0, t_0 = 0
for i = 0 TO m - 1 do
    p = (d · p + P[i]) (mod q)
    t_0 = (d · t_0 + T[i]) (mod q)
end for
for s = 0 TO n - m do
    if p == t_s then
        if P[0..m - 1] == T[s..s + m - 1] then
            print "Pattern occurs with shift " s
        end if
    end if
    if s < n - m then
        t_(s+1) = (d · (t_s - T[s] · h) + T[s + m]) (mod q)
    end if
end for
```

c) Step-by-step trace & Rolling Hash Mechanism (Without Modulo): To visually understand the core efficiency of Rabin-Karp, we first examine the sliding window technique and hash calculation without the modulo operation. Let us define a running example with Text $T = \texttt{cbabaca}$ and Pattern $P = \texttt{abaca}$ (where $m = 5$). For intuitive tracing, let the character values be $\texttt{a} = 1, \texttt{b} = 2, \texttt{c} = 3$, and choose the base $d = 10$.

The exact mathematical hash value of the pattern P is $p = 12131$. Figure 2 illustrates how the rolling hash updates in $O(1)$ time by dropping the most significant digit and appending a new least significant digit.

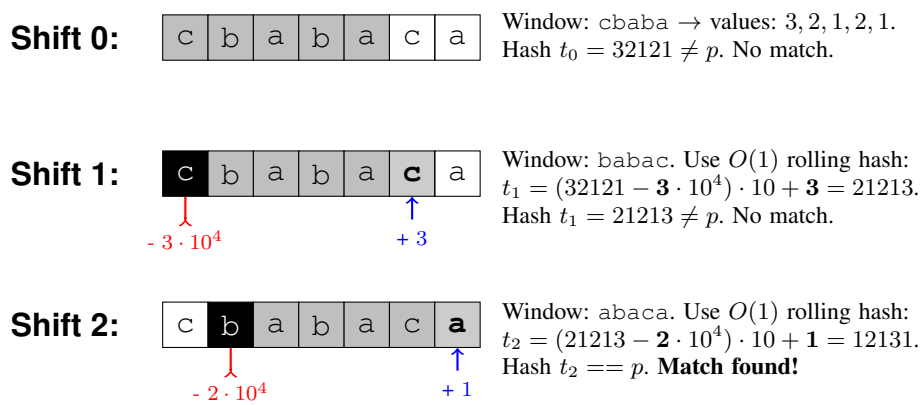

Fig. 2. Visual trace of the Rabin-Karp sliding window and rolling hash mechanism (using base $d = 10$ and no modulo). At each shift, the algorithm removes the value of the outgoing character (black cell) and adds the value of the incoming character (dark gray cell) to update the hash value in strictly $O(1)$ time.

d) The Overflow Problem & Choosing Parameters: While the exact hash computation works beautifully in concept, it is practically impossible to implement for long patterns. The mathematical hash value grows exponentially up to d^{m-1}. In C++, the largest standard integer type is `long long`, which is strictly limited to 8 bytes (64 bits) with a maximum capacity of $\approx 9 \times 10^{18}$.

The mathematical correctness and execution performance of the algorithm heavily depend on how we handle this overflow by choosing d and a modulo q:

- **Choosing the base d:** The base d must be strictly greater than the number of distinct characters in the alphabet. If $d \leq |\Sigma|$, two completely different strings could map to the exact same mathematical integer even before any modulo is applied, permanently ruining the uniqueness of the polynomial representation. Since the project's grid strictly contains lowercase English characters, we optimally choose $d = 26$.
- **The Modulo q & Spurious Hits:** If we search for a 20-character string using $d = 26$, the maximum exact hash value is $26^{19} \approx 7.6 \times 10^{26}$, which guarantees a catastrophic integer overflow. To prevent this, all arithmetic operations must be strictly performed modulo q. However, modular arithmetic introduces *spurious hits* (two different strings accidentally producing the same hash modulo q).
  Under the assumption of uniform hashing, the probability of a spurious hit is exactly $1/q$. By selecting q as a very large prime number, we uniformly distribute the hash values and minimize this collision rate, reducing the need for expensive $O(m)$ character-by-character verifications. Nevertheless, q must not be excessively large; the product $d \cdot q$ must still comfortably fit inside the 8-byte limit to avoid overflow during the intermediate $(d \cdot p + P[i])$ calculation step.

e) Complexity:

- **Time Complexity:** $\Theta(m)$ for preprocessing. The matching time is $O(n-m+1)$ in the best and average cases because hash collisions are exceptionally rare when q is large. However, in the absolute worst case (when spurious hits occur at every single shift), the secondary verification takes $O((n-m+1)m)$ time.
- **Space Complexity:** $O(1)$ since it only requires storing a few numerical variables (p, t_s, h) regardless of the text or pattern length.

f) Discussion: The Rabin-Karp algorithm is **good** (best-case scenario) when searching for multiple patterns simultaneously or when the hash function distributes values perfectly. If no hash collisions occur, the sliding window operates in $O(1)$ time per shift, resulting in a highly efficient $O(n+m)$ overall time.

However, it is strictly **bad** (worst-case scenario) when dealing with highly repetitive strings or severe hash collisions. Consider the exact same pathological example used in the Naïve approach: $T =$ `aaaa...aaa` and $P =$ `aaaaa`. In this scenario, the hash value of the pattern will perfectly match the

hash value of the text window at *every single shift*. Consequently, the algorithm is forced to perform the $O(m)$ character-by-character verification step repeatedly at every position, permanently losing its $O(1)$ hashing advantage and degrading the performance to the worst-case $O((n-m+1)m)$ time.

g) Conclusion: This algorithm is **good** when searching for multiple patterns simultaneously (by hashing all patterns and comparing the text's rolling hash against them in a Hash Set) or when dealing with large texts where the average processing time of $O(n)$ shines. It is **bad** in the worst-case scenario where the hash function causes many collisions (spurious hits), forcing the algorithm to repeatedly perform the expensive $O(m)$ string comparison.

h) Implementation Reference & Optimization: Our implementation of the Rabin-Karp algorithm is heavily based on the pseudocode and mathematical formulation from *Introduction to Algorithms* [1]. The code implements the standard rolling hash mechanism using modulo arithmetic to prevent integer overflow. Since this standard implementation already achieves the optimal $O(1)$ sliding window hash update, no non-standard structural optimizations were introduced.

3) Knuth-Morris-Pratt (KMP) Algorithm:

a) Introduction & Application: The Knuth-Morris-Pratt (KMP) algorithm improves upon the Naïve approach by guaranteeing a linear-time search even in the worst-case scenario. The core idea is to avoid re-evaluating characters of the text that have already been matched. To achieve this, the algorithm preprocesses the pattern P to compute a prefix function, often called the π array (or LPS - Longest Proper Prefix which is also Suffix). KMP is highly applicable in DNA sequencing, text editors, and systems where reading the text stream backward is impossible or costly.

Real-world Application: KMP is highly applicable in **DNA sequencing** in bioinformatics and network intrusion detection systems. Since it never backtracks the text pointer, it is the standard and most famous choice for parsing continuous data streams where reading the text backward is either impossible or too costly.

b) Pseudocode:

```
Function COMPUTE-PREFIX-FUNCTION(P)
m = P.length
let π[0..m − 1] be a new array
π = 0
k = 0
for i = 1 TO m − 1 do
   while k > 0 and P[k] ≠ P[i] do
      k = π[k − 1]
   end while
   if P[k] == P[i] then
      k = k + 1
   end if
   π[i] = k
end for
return π
Function KMP-MATCHER(T, P)
n = T.length, m = P.length
π = COMPUTE-PREFIX-FUNCTION(P)
q = 0
for i = 0 TO n − 1 do
   while q > 0 and P[q] ≠ T[i] do
      q = π[q − 1]
   end while
   if P[q] == T[i] then
      q = q + 1
```

```
    end if
    if q == m then
        print "Pattern occurs with shift " i − m + 1
        q = π[q − 1]
    end if
end for
```

c) Understanding the Fallback Mechanism ($k = \pi[k-1]$): The most confusing yet brilliant part of KMP is the fallback logic inside the `while` loop: $k = \pi[k-1]$ (or $q = \pi[q-1]$ during matching). When a mismatch occurs at $P[k] \neq P[i]$, it means we cannot extend the current matching prefix of length $k+1$. However, we *already know* that the previous k characters matched perfectly. Instead of resetting our search pointer k entirely back to 0 and doing redundant comparisons, we want to find the *next longest proper prefix* of $P[0..k-1]$ that is also a suffix of $P[0..k-1]$.

By definition, the value $\pi[k-1]$ stores exactly the length of this next-best matching prefix! By assigning $k = \pi[k-1]$, we gracefully slide the pattern to the next valid prefix-suffix alignment and try comparing the new $P[k]$ with $P[i]$ again. If it still mismatches, we recursively cascade the fallback ($k = \pi[k-1]$) until a match is found or k reaches 0.

d) Step-by-step trace: To graphically demonstrate the internal states and the powerful fallback mechanism, we trace the execution on a new running example with $T =$ `bacbababaabcbab` and $P =$ `ababaca`.

1. Preprocessing Phase (π array computation):
figure* 3 visualizes how the pattern is shifted against itself. The most notable step is at $i = 5$, where a cascade of fallbacks occurs because `'c'` fails to match the expected characters.

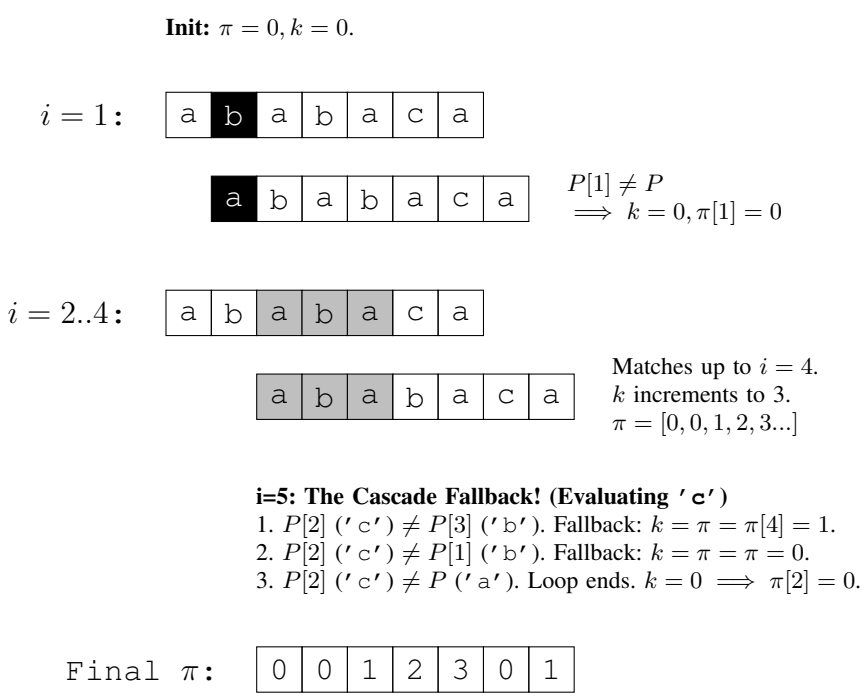

Fig. 3. Visual trace of the π array computation for $P =$ `ababaca`. Notice the fallback cascade at $i = 5$.

2. Matching Phase:
figure* 4 visualizes the search. We specifically highlight Shift 4, where the algorithm perfectly utilizes the fallback cascade to shift the window without completely resetting the text pointer i.

e) Complexity:

- **Time Complexity:** $\Theta(m)$ for preprocessing the π array and $\Theta(n)$ for the matching phase. The overall time complexity is $\Theta(n+m)$. Even though there is a `while` loop inside the `for` loop, an amortized

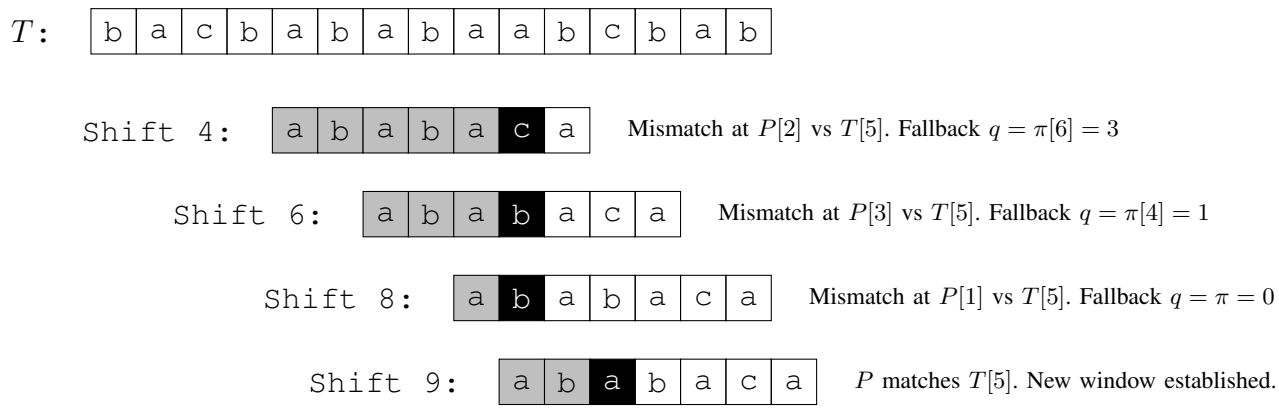

Fig. 4. Visual trace of the KMP matching phase. At Shift 4, P matched 5 characters but failed at $T[5] = \texttt{'a'}$. KMP dynamically falls back $q \to 3 \to 1 \to 0$ to safely slide the pattern without reversing the text pointer i.

analysis shows that q (or k) can be decreased at most as many times as it is increased, bounding the total number of while-loop iterations to n.

- **Space Complexity:** $\Theta(m)$ to store the prefix array π of length m.

f) Discussion: The KMP algorithm is strictly **good** and highly resilient exactly when the Naïve and Rabin-Karp algorithms perform at their worst. In the worst-case scenario for Naïve ($T = \texttt{aaaa...aaa}$ and $P = \texttt{aaaaa}$), KMP flawlessly utilizes its π array to intelligently shift the pattern without ever backtracking the text pointer i. It guarantees a strict $O(n)$ time bound regardless of how repetitive the input is.

However, it can be considered **bad** (or rather, suboptimal) in the average best-case scenario for standard English text searching. Unlike Boyer-Moore, KMP is forced to inspect every single character in the text T at least once. Therefore, its absolute best-case running time is bounded by $\Omega(n)$, making it practically slower than algorithms capable of sublinear skipping.

g) Conclusion: The KMP algorithm is strictly **good** when the text contains many repetitive overlapping patterns (e.g., $T = \texttt{aaaaa...}$, $P = \texttt{aaa}$) or when scanning an un-rewindable data stream. In such worst-case scenarios for the Naïve algorithm, KMP flawlessly maintains an $O(n)$ bound by gracefully shifting its pointer using the π array. However, it can be considered **suboptimal** in practical standard English text searching compared to algorithms like Boyer-Moore, because KMP always inspects every single character in the text at least once, whereas Boyer-Moore can skip characters entirely.

h) Implementation Reference & Optimization: The source code for the KMP algorithm, including the preprocessing phase to build the prefix function (π array), is referenced directly from *Introduction to Algorithms* [1]. The code flawlessly translates the standard optimal $O(n+m)$ algorithm into C++. Because the textbook's logic is mathematically proven to be optimal by avoiding text pointer backtracking entirely, no further custom optimizations were necessary.

4) Boyer-Moore Algorithm:

a) Introduction & Application: The Boyer-Moore algorithm is widely considered the benchmark for the most efficient string-matching algorithm in practical applications. Unlike the Naïve or KMP algorithms, which scan the pattern from left to right, Boyer-Moore elegantly compares characters from **right to left**.

Based on our specific C++ implementation, this approach heavily relies on the *Bad Character Heuristic* to find all occurrences of a pattern. The core idea is brilliant: when a mismatch occurs between the pattern $P[j]$ and the text $T[s+j]$, we look at the mismatched "bad character" in the text. If this character exists in the pattern, we shift the pattern so that its last occurrence aligns with the text. If it doesn't exist, we can confidently slide the pattern entirely past this character. Furthermore, our implementation contains a powerful optimization: when a full match is found, instead of simply shifting by 1, it examines the

very next character in the text ($T[s+m]$) and utilizes the bad character table to jump ahead, drastically reducing comparisons in repetitive texts.

b) Pseudocode: The following pseudocode accurately reflects our group's C++ implementation using a 0-based array indexing format.

```
Function BOYER-MOORE-MATCHER(T, P)
n = T.length, m = P.length
let last[0..25] be a new array initialized to −1 {For lowercase English alphabet}
for i = 0 TO m − 1 do
  last[P[i] − 'a'] = i
end for
let res be an empty list
s = 0
while s ≤ n − m do
  j = m − 1
  while j ≥ 0 and P[j] == T[s + j] do
    j = j − 1
  end while
  if j < 0 then
    append s to res {A full match is found}
    if s + m < n then
      s = s + m − last[T[s + m] − 'a'] {Smart jump after a match}
    else
      s = s + 1
    end if
  else
    s = s + max(1, j − last[T[s + j] − 'a']) {Shift on mismatch}
  end if
end while
return res
```

c) Understanding the Shift Logic:

- **Mismatch Shift:** When a mismatch occurs at index j, we shift by $\max(1, j - last[T[s+j]])$. The max function acts as a safety net. If the mismatched character's last occurrence in P is actually to the *right* of j, the simple difference would be negative (moving the pattern backward). The $\max(1, \dots)$ ensures the pattern always progresses by at least 1 position.
- **Match Shift:** When a full match is found ($j < 0$), we don't just shift by 1. We look at the character just beyond the current window, $T[s+m]$, and align the pattern's last occurrence of that character with it, allowing massive skips even immediately after a success.

d) Step-by-step trace: To graphically demonstrate the power of our specific code, we trace its execution on a highly repetitive running example: Text $T = \texttt{aabacabacababacc}$ ($n = 16$) and Pattern $P = \texttt{abacab}$ ($m = 6$).

Preprocessing: The $last$ array records the rightmost index of each character in P: $last[\texttt{a}] = 4$, $last[\texttt{b}] = 5$, $last[\texttt{c}] = 3$. All other characters are -1.

e) Complexity:

- **Time Complexity:** The preprocessing of the $last$ array strictly takes $\Theta(m + |\Sigma|)$ time. In the worst-case scenario (e.g., $T = \texttt{aaaaa}$, $P = \texttt{baaa}$), the running time degrades to $O(nm)$ because the bad character heuristic alone cannot avoid repetitive comparisons. However, in the average and best cases (typical English text), the time complexity is remarkably sublinear at $O(n/m)$ due to massive jumps.
- **Space Complexity:** $O(|\Sigma|)$ to store the $last$ occurrence array. Because our problem strictly guarantees lowercase English alphabets ($|\Sigma| = 26$), the auxiliary space footprint is absolutely $O(1)$, which is

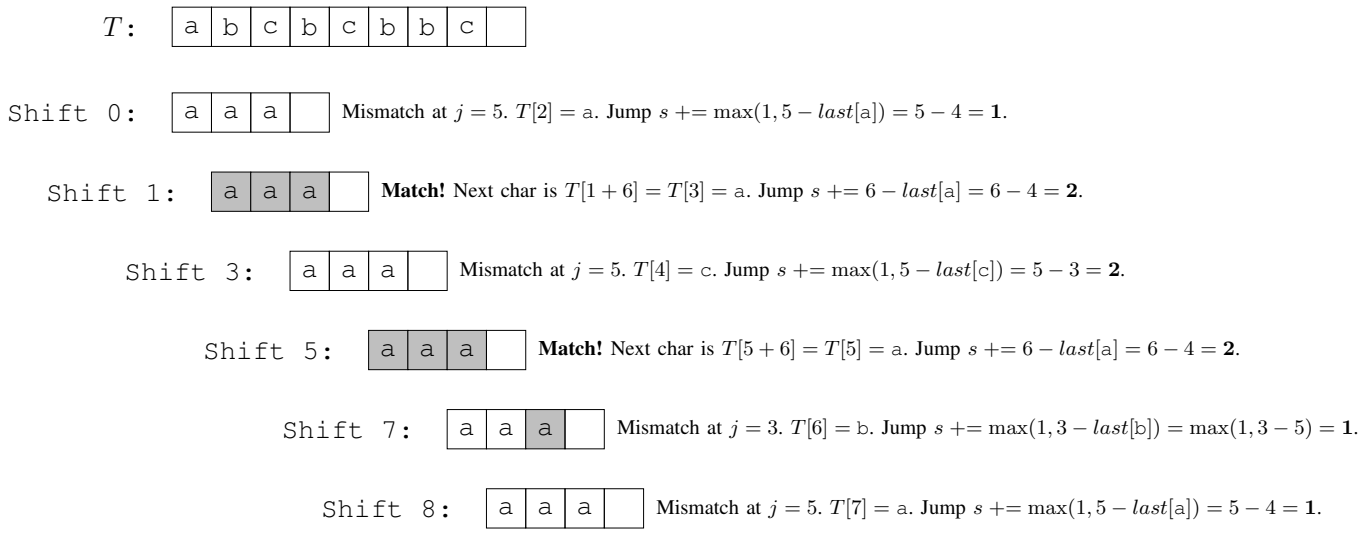

Fig. 5. Visual trace of our Boyer-Moore implementation. Notice two critical behaviors: (1) At Shift 1 and 5, after finding a full match, the algorithm smartly inspects the next text character to skip redundant checks. (2) At Shift 7, the mismatch occurs at $j = 3$ against $T[6] = \mathtt{b}$. Since $last[\mathrm{b}] = 5 > 3$, the formula $3 - 5$ yields -2. The $\max(1, -2)$ logic safely prevents the pattern from shifting backward, forcing a forward shift of 1.

highly memory-efficient.

f) Discussion: Our implemented version of Boyer-Moore is exceptionally **good** (best-case scenario) in practical string matching with large alphabets (like English text) and long patterns. If the text character causing the mismatch does not exist in the pattern at all (e.g., $T = \mathtt{xyzxyzxyz}$, $P = \mathtt{abc}$), the bad character heuristic allows the algorithm to skip m characters entirely. This results in a remarkable sublinear best-case running time of $O(n/m)$.

Conversely, it is **bad** (worst-case scenario) for extremely small alphabets (like binary `0`s and `1`s or DNA sequences) or pathological inputs. For example, if $T = \mathtt{aaaa...aaa}$ and $P = \mathtt{baaaa}$, the mismatch occurs at the rightmost character. Since `'a'` is heavily present in P, the formula $\max(1, j - last)$ triggers the safety net, forcing the pattern to shift by merely 1 step each time. Without the Good Suffix heuristic, our implementation degrades to $O(nm)$ time, similar to the Naïve algorithm.

Real-world Application: Boyer-Moore is famous for being the engine behind specific system commands like **GNU `grep`** in Unix/Linux and the standard **"Find"** tool in modern text editors. Its right-to-left scanning and massive skipping heuristics make it the undisputed benchmark for practical string matching in standard text.

g) Conclusion: Our implemented version of Boyer-Moore is exceptionally **good** in practical string matching, especially for long patterns and large alphabets. The counter-intuitive beauty of this implementation is that *the longer the pattern is, the faster the algorithm performs*, because the potential maximum jump size m increases! Furthermore, the added heuristic that calculates shifts based on $T[s + m]$ after a successful match elevates its performance over standard textbook versions. However, it can be **suboptimal** for extremely small alphabets (like binary `0`s and `1`s or DNA sequences). In such cases, mismatches often align with characters that appeared very recently in the pattern, triggering the $\max(1, j - last)$ safety net repeatedly and degrading the shift size to a mere 1, similar to the Naïve algorithm.

h) Implementation Reference & Optimization: The Boyer-Moore implementation is referenced from the algorithmic logic and code structures provided by GeeksforGeeks [2], Number Analytics [3], and CCBP [4]. Our code primarily implements the standard Bad Character Heuristic using a `last` occurrence array of size 256 (for the full ASCII set) to perform sublinear skips. Since this strictly follows the standard heuristic logic for practical text searching, it is highly optimized by default. No further deviations from the standard algorithm were applied.

5) Aho-Corasick Algorithm:

a) Introduction & Application: While Brute-Force, KMP, and Boyer-Moore are designed to search for a single pattern at a time, the Aho-Corasick algorithm is a highly efficient **multiple-pattern** string-matching algorithm. It combines the Trie data structure with a failure-link mechanism (conceptually similar to the KMP prefix function). The algorithm constructs a finite-state automaton from a set of keywords in the preprocessing phase. During the search phase, it scans the text exactly once in linear time. When a mismatch occurs, instead of restarting the search, the algorithm gracefully transitions to the longest valid suffix using failure links, allowing it to detect multiple patterns simultaneously.

Real-world Application: Because of its unmatched ability to search for thousands of keywords concurrently within a single pass, Aho-Corasick is famously known for its core role in **antivirus software, spam filtering, and network intrusion detection systems** (such as Snort), where incoming data streams must be matched against a massive database of virus signatures or forbidden words.

b) Pseudocode: Based on our implementation, the algorithm consists of building the state machine and then searching the text.

```
Function BUILD-MATCHING-MACHINE(keywords)
Initialize a Trie with a root node (state 0)
for each word P in keywords do
    Insert P into the Trie. Mark the end state with the word index.
end for
Initialize a queue Q for BFS to build failure links
for each character c attached to root do
    Set failure link of child state to root (0)
    Enqueue child state
end for
while Q is not empty do
    u = Dequeue(Q)
    for each child state v of u with character c do
        fail = failure_link[u]
        while fail ≠ 0 and no child c from fail do
            fail = failure_link[fail]
        end while
        failure_link[v] = child of fail with char c (or 0)
        Merge output list of failure_link[v] into v's output list
        Enqueue v
    end for
end while
return the automaton
Function AHO-CORASICK-SEARCH(T, keywords)
n = T.length, state = 0
let res be an empty list of matches
for i = 0 TO n − 1 do
    while state ≠ 0 and no transition for T[i] from state do
        state = failure_link[state]
    end while
    state = transition from state using T[i] (or 0)
    if state has outputs then
        for each word P in output list of state do
            append (i − P.length + 1) to res
        end for
    end if
```

end for
return res

c) Step-by-step trace: **1. Preprocessing:** A Trie is built for the keywords. Failure links are established and output links are merged. Figure 6 illustrates the complete state machine.

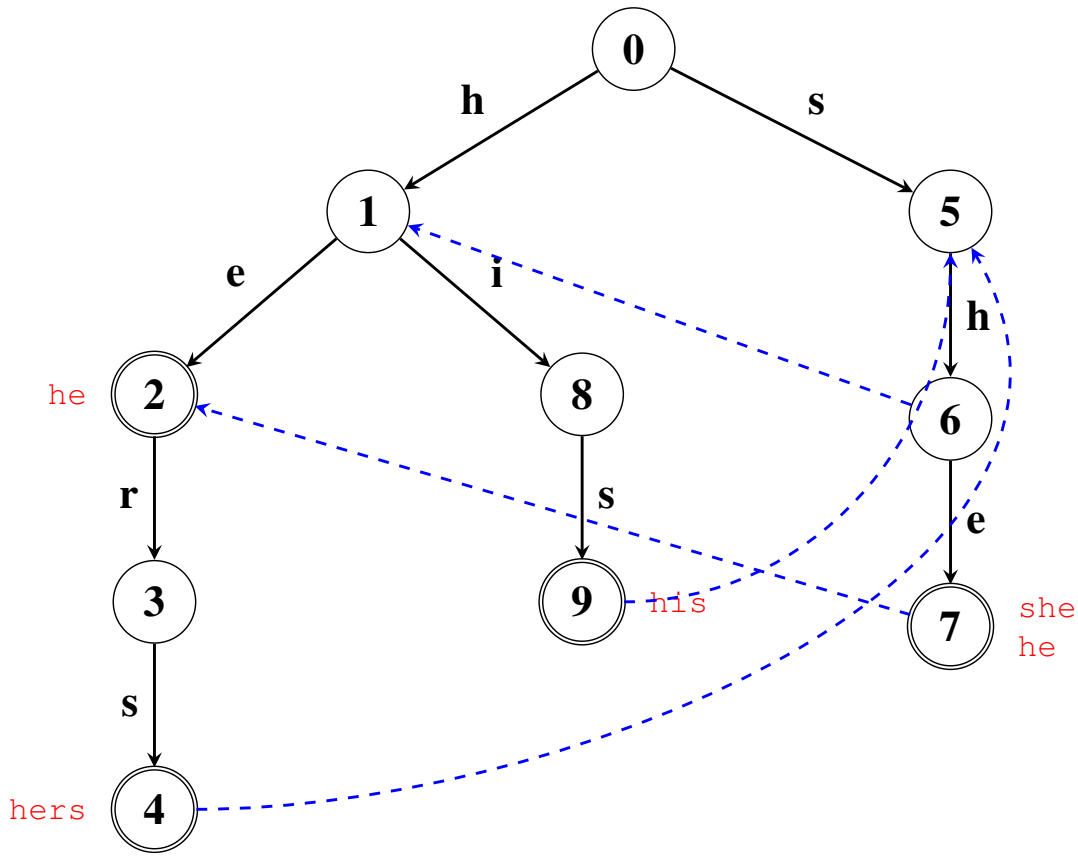

Fig. 6. The Aho-Corasick Automaton for the dictionary {"he", "she", "his", "hers"}. Solid lines represent standard Trie transitions. Dashed blue lines denote failure links (for visual clarity, failure links pointing directly to the root state 0 are omitted). Nodes with double circles are accepting states, annotated with their matched patterns. Notice how state 7 merges the output of state 2 via its failure link.

2. Searching (Character-by-character scan):
The algorithm scans the text $T =$ `ahishers` from left to right ($i = 0$ to 7). At each step, it updates the current state according to the transition function or falls back using failure links.

- **Step 1** ($i = 0$, **`'a'`**): Current state is 0. No transition for `'a'`. State remains 0. No pattern matched.
- **Step 2** ($i = 1$, **`'h'`**): Transition from state $0 \xrightarrow{\text{'h'}} 1$. No pattern matched.
- **Step 3** ($i = 2$, **`'i'`**): Transition from state $1 \xrightarrow{\text{'i'}} 8$. No pattern matched.
- **Step 4** ($i = 3$, **`'s'`**): Transition from state $8 \xrightarrow{\text{'s'}} 9$. State 9 is an accepting state.
  Match found: `"his"` at position $3 - 3 + 1 = \mathbf{1}$.
- **Step 5** ($i = 4$, **`'h'`**): State 9 has no transition for `'h'`. The algorithm follows the failure link from state 9 to state 5 (representing prefix `"s"`). From state 5, it transitions $5 \xrightarrow{\text{'h'}} 6$. No pattern matched.
- **Step 6** ($i = 5$, **`'e'`**): Transition from state $6 \xrightarrow{\text{'e'}} 7$. State 7 is an accepting state representing `"she"`. Due to dictionary merging (output link from 7 to 2), pattern `"he"` is also detected simultaneously.
  Match found: `"she"` at position $5 - 3 + 1 = \mathbf{3}$.
  Match found: `"he"` at position $5 - 2 + 1 = \mathbf{4}$.
- **Step 7** ($i = 6$, **`'r'`**): State 7 has no transition for `'r'`. It follows the failure link from 7 to state 2 (representing prefix `"he"`). From state 2, it transitions $2 \xrightarrow{\text{'r'}} 3$. No pattern matched.

- **Step 8 ($i = 7$, `'s'`):** Transition from state $3 \xrightarrow{\text{`s'}} 4$. State 4 is an accepting state.
  Match found: `"hers"` at position $7 - 4 + 1 = \mathbf{4}$.

Figure 6 visually summarizes this matching sequence on the text array.

d) Complexity: Let n be the length of the text, z be the total number of matches found, m_i be the length of the i-th pattern, and $|\Sigma|$ be the size of the alphabet.

- **Time Complexity:** Building the automaton takes $O(\sum m_i)$ time. Searching the text takes $O(n + z)$ time. The overall time complexity is precisely $O(n + \sum m_i + z)$, which is strictly linear and optimal for multiple pattern matching.
- **Space Complexity:** $O(\sum m_i \times |\Sigma|)$ to store the Trie, the failure links, and the output links.

e) Discussion: The Aho-Corasick algorithm is strictly **good** (best-case scenario) when we need to search for a massive dictionary of thousands of keywords simultaneously (e.g., 100×100 grid with 500 keywords). In this scenario, algorithms like Naïve, KMP, or Boyer-Moore would have to rescan the entire text 500 times, whereas Aho-Corasick gracefully finds all 500 patterns in exactly one single linear scan.

However, it can be considered **bad** (worst-case scenario) when searching for only one single pattern or when memory is strictly limited. The overhead of allocating states, building the Trie, and establishing BFS failure links takes significant time and memory. If the dictionary is small, the sublinear skipping power of Boyer-Moore usually dominates Aho-Corasick in practical standard text searching.

f) Implementation Reference & Optimization: The source code for the Aho-Corasick algorithm is synthesized and heavily referenced from tutorials by Programming Algorithms [5], GeeksforGeeks [6], and the VNOI Wiki [7].

Applied Optimization: While traditional implementations of the Aho-Corasick Automaton rely on dynamically allocated memory and pointers to construct the Trie nodes, our implementation introduces a significant memory-management optimization. Instead of pointers, we utilize pre-allocated static 1D and 2D arrays (`Out[MaxStates]`, `FF[MaxStates]` for Failure Links, and `GF[MaxStates][MaxChars]` for the Goto Function/Trie edges). This array-based automaton completely avoids the overhead and memory fragmentation caused by dynamic allocation (`new` or `malloc`). Consequently, it dramatically improves CPU cache locality and accelerates the state transition process during the search phase.

IV. Experimental Results

A. Experimental Results

To empirically evaluate the performance of the implemented string-matching algorithms, we benchmarked them according to the two required scenarios: varying the grid size and varying the dictionary size. Note that our advanced algorithm, Aho-Corasick, is also included in the comparison. The patterns consist of fully random characters, and their lengths are randomly generated in the range of $[1, 5]$. The execution time is measured strictly for the searching phase in milliseconds (ms).

a) Scenario 1: Impact of Grid Size: In this scenario, we fixed the number of keywords to $K = 10$ and increased the grid size from 10×10 to 500×500.

TABLE II
Execution Time (ms) - Fixed 10 Keywords, Varying Grid Size

Grid Size	Keys	Naïve	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
10×10	10	0.00 ms	0.00 ms	0.00 ms	0.00 ms	0.00 ms
100×100	10	14.57 ms	0.00 ms	16.47 ms	0.00 ms	0.00 ms
500×500	10	78.18 ms	164.14 ms	109.48 ms	117.47 ms	23.07 ms

b) Scenario 2: Impact of Dictionary Size: In this scenario, we fixed the grid size to 100×100 and varied the number of searched keywords ($K = 10, 100, 500$).

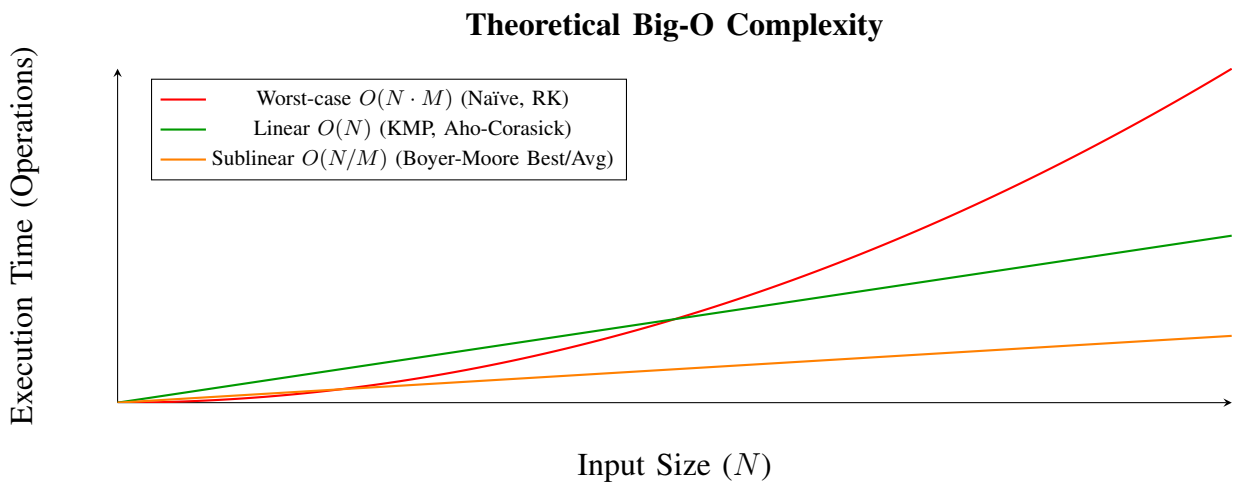

Fig. 7. Theoretical order of growth for the string-matching algorithms.

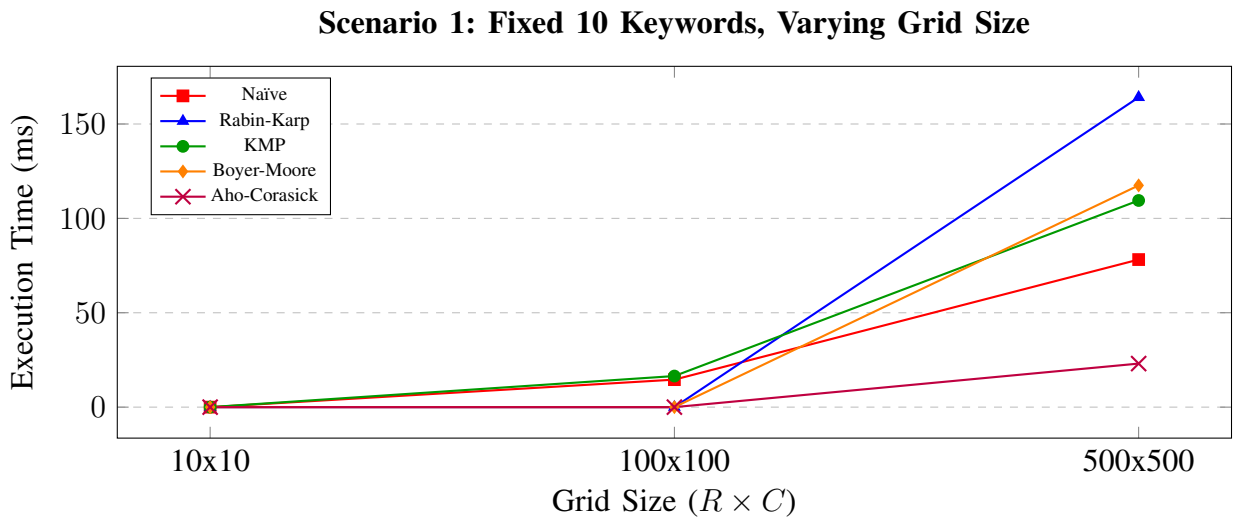

Fig. 8. Line chart illustrating the execution time (ms) as the grid size scales up. Notice how Aho-Corasick maintains the lowest execution time for large grids.

c) Performance Evaluation & Comments: Based on the experimental data presented above, we address the required analytical questions:

- The growth rate generally matches the theoretical time complexity analyzed earlier. As the grid size increases (e.g., from 100×100 to 500×500), the execution time of all algorithms increases significantly because the amount of text that must be scanned becomes much larger. Since the preprocessing phase is not included in the measurements, the results mainly reflect the search phase complexity, which is directly proportional to the size of the text being scanned.

TABLE III
EXECUTION TIME (MS) - FIXED 100×100 GRID, VARYING KEYWORD COUNT

Grid Size	Keys	Naïve	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
100×100	10	14.57 ms	0.00 ms	16.47 ms	0.00 ms	0.00 ms
100×100	100	38.07 ms	45.99 ms	57.87 ms	38.52 ms	15.57 ms
100×100	500	200.59 ms	274.37 ms	333.19 ms	205.70 ms	203.79 ms

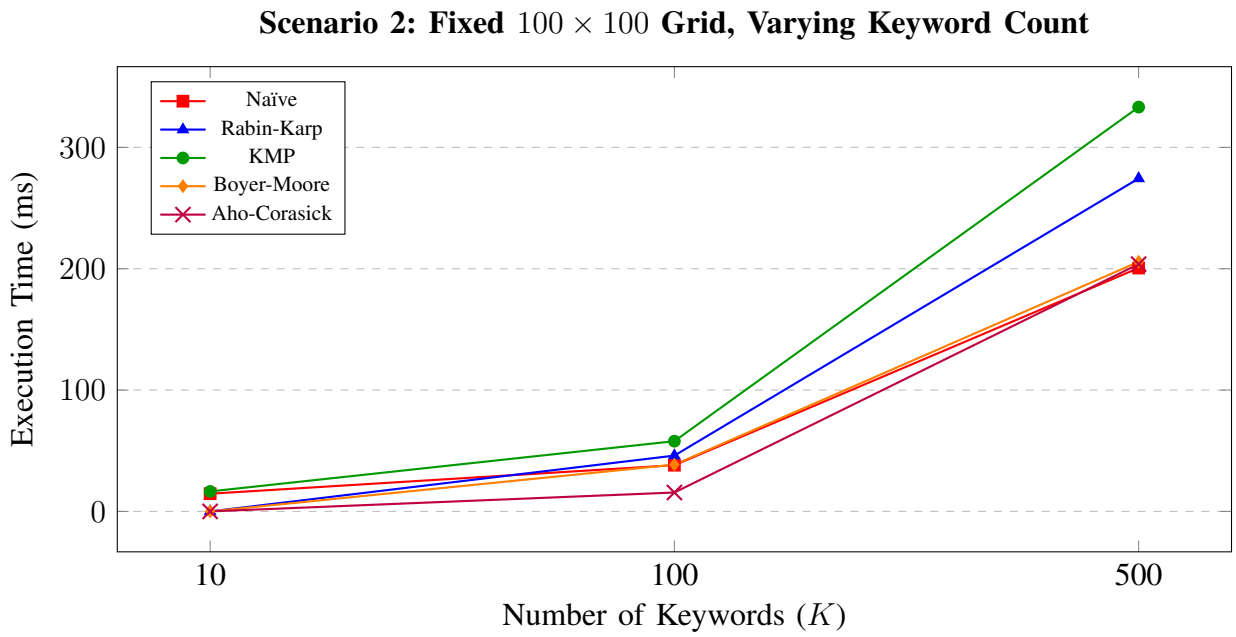

Fig. 9. Line chart illustrating the execution time (ms) as the number of keywords increases from 10 to 500 within a fixed 100×100 grid. Observe the counter-intuitive efficiency of the Naïve algorithm compared to KMP and Rabin-Karp due to extremely short pattern lengths.

- For large grids with a small number of keywords, **Aho-Corasick performs the best**, as clearly shown in the 500×500 grid with 10 keywords (23.07 ms), where it is significantly faster than all other algorithms. This is because Aho-Corasick scans the text only once while checking all patterns simultaneously.
  Surprisingly, there are situations where the **Brute Force (Naïve)** approach performs similarly to or even faster than optimized algorithms. On the 500×500 grid with 10 keywords, Naïve (78.18 ms) easily outperformed Rabin-Karp, KMP, and Boyer-Moore. Furthermore, on the 100×100 grid with 500 keywords, Naïve (200.59 ms) narrowly beat Aho-Corasick (203.79 ms). Since preprocessing time is excluded, the advantage of algorithms like KMP or Rabin-Karp becomes smaller, making the simple character-by-character comparison of Brute Force highly competitive.
- The results are mainly influenced by two critical factors from our test data:
  1) **Very Short Pattern Lengths:** The pattern lengths in our test cases are extremely small (1–5 characters). With such short patterns, mismatches occur very quickly, which drastically reduces the benefit of sophisticated skipping strategies used in algorithms such as KMP or Boyer-Moore.
  2) **Measurement Scope (Search Phase Only):** The experiment measures only the searching phase, ignoring the preprocessing phase. As a result, algorithms that require heavy preprocessing (such as building prefix tables for KMP or automata for Aho-Corasick) do not show their full theoretical overhead in the measurements. This makes the practical differences between algorithms smaller, particularly for small inputs.

V. References

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] GeeksforGeeks, "Boyer-Moore Algorithm for Pattern Searching." [Online]. Available: https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/. [Accessed: Mar. 1, 2026].

[3] Number Analytics, "Explaining Boyer-Moore algorithm." [Online]. Available: https://www.numberanalytics.com/blog/explaining-boyer-moore-algorithm. [Accessed: Mar. 8, 2026].

[4] Nxt Wave, "Boyer-Moore Algorithm." [Online]. Available: https://www.ccbp.in/blog/articles/boyer-moore-algorithm. [Accessed: Mar. 4, 2026].
[5] Programming Algorithms, "Aho-Corasick algorithm in C++." [Online]. Available: https://www.programmingalgorithms.com/algorithm/aho%E2%80%93corasick-algorithm/cpp/. [Accessed: Mar. 9, 2026].
[6] GeeksforGeeks, "Aho-Corasick Algorithm for Pattern Searching." [Online]. Available: https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/. [Accessed: Mar. 9, 2026].
[7] VNOI, "Aho-Corasick." [Online]. Available: https://wiki.vnoi.info/vi/algo/string/aho-corasick. [Accessed: Mar. 1, 2026].

VI. Acknowledgment

Our group would like to express our deepest gratitude to our instructors for their continuous guidance and valuable feedback throughout this project. In addition, AI tools, such as ChatGPT and NotebookLM, assisted with drafting sections of the report in academic English, explaining the core concepts of advanced string-matching algorithms, and providing examples for the code implementations.